

Production-Grade Agentic AI for High-Stakes Enterprise Workflows

*A Reliability, Evaluation, and Accountability Framework for Moving Agents
from Demo to Production*

**Autonomy without recoverability, evaluation, and accountability
is a liability.**

Author: Shiladitya Srivastava
Principal Software Engineer & Technical Architect
September 2026 · Version 1.0

Abstract

Large language model (LLM) agents now demonstrate remarkable capability, yet the systems that reach production reliably remain a small fraction of those that demo well. The gap is not primarily one of intelligence; it is one of engineering. Agents that take many dependent steps against real systems accumulate error, mutate external state, and fail in ways that no benchmark captures. This paper argues that the defining discipline of production-grade agentic AI is not raw capability but *recoverability, measurability, and accountability*. It contributes a two-axis Agentic Maturity Model that pairs levels of autonomy with independently maturing capability dimensions; a layered reference architecture whose center of gravity is a reliability and learning control plane; a catalog of reliability patterns, including reliability budgeting; and a closed-loop model for self-healing and continuous learning. Throughout, a single archetype — multi-step enterprise onboarding and provisioning — grounds the abstractions in a scenario every enterprise recognizes. The framework is vendor-neutral and draws on both established distributed-systems practice and recent empirical evidence on long-horizon agent degradation and production-grade evaluation.

Contents

Abstract..... 2

Contents..... 3

1. Executive Summary 4

2. The Tension: When an Agent Breaks a Tenant 4

3. Why Production Agentic AI Is Hard..... 5

4. The Agentic Maturity Model 5

 4.1 Axis A — Levels of Autonomy 6

 4.2 Axis B — Capability Dimensions..... 7

5. A Reference Architecture for Production Agents 7

6. The Reliability Pattern Catalog 9

 6.1 Reliability Budgeting 9

7. Quality and Evaluation: The Measurement Backbone 10

8. Accountability and Auditability 10

9. Learning from Production: Reinforcement Learning Done Safely 11

10. Self-Healing and Semi-Autonomous Recovery 11

11. Anti-Patterns and Trade-offs: When Not To.....12

12. A Worked Example: Onboarding from L1 to L4.....13

13. Getting Started: A Maturity Self-Assessment.....14

14. The Companion Series: Building Agents That Ship15

Selected References16

1. Executive Summary

Every organization building with LLM agents has felt the same whiplash: a prototype that dazzles in a demo, followed by a production deployment that quietly fails on the third customer, the tenth step, or the second week. The uncomfortable truth is that capability demos are easy and production reliability is hard — and the two are governed by different disciplines.

This paper makes one central claim: **autonomy without recoverability, evaluation, and accountability is a liability**. An agent that can act but cannot recover, cannot be measured, and cannot be held to account is not an asset in a high-stakes workflow; it is a risk multiplier. The more autonomy such an agent is granted, the larger the blast radius of its inevitable mistakes.

To turn that claim into engineering practice, the paper offers four contributions:

1. An **Agentic Maturity Model** with two axes — a familiar autonomy ladder (L0–L5) and, orthogonal to it, five capability dimensions (reliability, evaluation, accountability, learning, self-healing) that each mature on their own timeline. The governing rule: autonomy must never exceed the weakest capability dimension.
2. A **reference architecture** whose organizing idea is that the reliability, observability, and learning control plane — not the model — is the load-bearing structure of a production agent.
3. A **reliability pattern catalog**, including *reliability budgeting*: sizing the per-step reliability a workflow needs before committing to a horizon.
4. A **closed-loop model** for self-healing and continuous, reinforcement-driven improvement that treats recovery and learning as system properties rather than human chores.

The intended reader is a senior engineer, architect, or engineering leader responsible for taking agents from proof-of-concept to dependable production. The takeaways are deliberately vendor-neutral so they survive the next model release and generalize beyond any one platform or organization.

This paper is also the anchor of a companion series, *Building Agents That Ship*, that turns each capability dimension into a standalone, hands-on installment. Section 14 maps the two together, so the framework here and the practice there can be read as one.

2. The Tension: When an Agent Breaks a Tenant

Consider a concrete, if anonymized, scenario. An agent is asked to onboard a new enterprise customer: create identities, assign licenses, configure domains and DNS, and migrate a first tranche of data. In a demo, it sails through all ten steps. In production, on a real tenant, it fails at step seven — a licensing call returns an unexpected state — and stops. It has already created identities and altered DNS. It has not rolled anything back, because nothing told it how. A human now inherits a half-configured tenant that no person actually touched, and must reverse-engineer what the agent did before they can safely continue.

This failure mode is nearly invisible in the way we usually evaluate agents. Benchmarks score final outputs; they rarely score the *side effects an agent leaves on the world* when it fails midway. Yet in enterprise workflows, those side effects are the whole game. The reframing that follows from this scenario is the intellectual spine of the paper:

Onboarding a customer is a distributed transaction, not a chat.

Once we see the workflow as a distributed transaction — a sequence of state-mutating operations that must either complete or be safely unwound — decades of hard-won distributed-systems practice become directly relevant. The problem is not that the agent is not smart enough. The problem is that the *system around the agent* provides none of the transactional guarantees the task actually requires.

3. Why Production Agentic AI Is Hard

Three structural properties make production agents fundamentally harder than the single-shot LLM calls that preceded them.

Non-determinism. The same input can produce different trajectories. This is a feature for creativity and a liability for operations, where reproducibility underpins testing, debugging, and trust.

Compounding error. An agent that succeeds at each step with probability r succeeds at an H -step task with probability that falls off geometrically as r raised to the power H . Recent empirical work puts hard numbers on this intuition: across a large controlled study of many models and thousands of trajectories, agent task success followed a geometric law governed by a single per-step reliability parameter that rises with model scale but saturates well below one — guaranteeing eventual collapse at sufficiently long horizons.¹ On genuinely agentic tool-use tasks, even strong, widely deployed systems fell from near-perfect to near-zero within roughly a dozen and a half steps. Two implications matter for architects. First, aggregate benchmark pass-rates systematically overstate production reliability, because production horizons are an order of magnitude longer. Second — counter-intuitively — aggressively truncating context to save cost *steepens* the decay rather than easing it, because degradation is driven by step count, not context length. The popular shortcut makes the problem worse.

Unbounded action space. Unlike a chatbot, an agent invokes tools that change real systems: identity providers, license managers, DNS, data stores. Every tool is both a capability and a hazard.

Layered on top of these are the four things demos routinely skip — failure, state, scale, and security — and the three disciplines teams routinely under-invest in: evaluation, accountability, and learning. The rest of this paper is organized around closing exactly those gaps. The one-sentence takeaway: **quality is engineered, not prompted, and reliability must be budgeted per horizon.**

4. The Agentic Maturity Model

Progress toward dependable agents is easier to reason about when we separate two questions that are usually conflated: *how much autonomy* an agent has, and *how mature the capabilities* are that make that autonomy safe. The model therefore has two axes.

¹“How Fast Do Agents Rot?” arXiv:2609.01660, 2026 (recent preprint). <https://arxiv.org/abs/2609.01660>

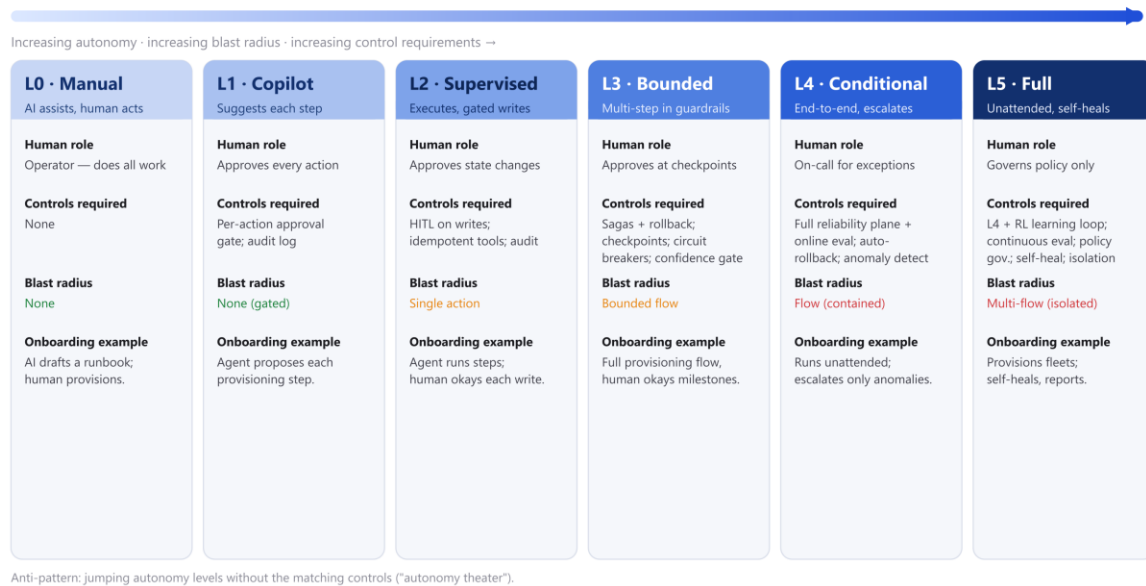
4.1 Axis A — Levels of Autonomy

Borrowing, by analogy, from the graded levels used in driving automation,² we describe six levels of agent autonomy. Each level is defined not only by what the agent does but by the control plane it *requires* to do it safely.

Autonomy Maturity Model for Enterprise Agents

Axis A: autonomy L0–L5. Each level demands a matching control plane (see Axis B: Reliability, Evaluation, Accountability, Learning, Self-Healing).

Thesis: autonomy without recoverability is a liability.



Anti-pattern: jumping autonomy levels without the matching controls ("autonomy theater").

Figure 1. Axis A — autonomy levels L0–L5 and the controls each level demands.

Level	Definition	Human role	Controls required	Blast radius
L0 Manual	AI assists, human acts	Operator	None	None
L1 Copilot	Suggests each step	Approves every action	Per-action gate; audit log	None
L2 Supervised	Executes; writes gated	Approves state changes	HITL on writes; idempotent tools; audit	Single action
L3 Bounded	Multi-step within guardrails	Approves at checkpoints	Sagas + rollback; checkpoints; circuit breakers; confidence gating	Bounded flow
L4 Conditional	End-to-end; escalates	On-call for exceptions	Full reliability plane; online eval; auto-rollback; anomaly detection	Flow (contained)
L5 Full	Unattended; self-heals	Governs policy only	L4 + continuous eval; RL loop; policy governance; blast-radius isolation	Multi-flow (isolated)

²SAE International, "J3016: Levels of Driving Automation" (analogy only).
https://www.sae.org/standards/content/j3016_202104/

4.2 Axis B — Capability Dimensions

Autonomy is only half the picture. Five capabilities determine whether a given level of autonomy is *earned* or reckless. Crucially, each matures on its own timeline, from ad hoc to optimizing.

Agentic Maturity — Axis B: Capability Dimensions

Each dimension matures independently (1→5). Governing rule: autonomy (Axis A) must never exceed your weakest dimension here.

	1 · Ad hoc reactive	2 · Repeatable documented	3 · Managed controlled	4 · Measured quantified	5 · Optimizing self-improving
Reliability safe failure & recovery	best-effort; no rollback	retries + idempotent tools	sagas, checkpoints, circuit breakers	SLOs + tested failure; blast- radius controls	self-healing; blast isolation
Evaluation / Quality measurement backbone	manual spot- checks ("vibes")	fixed test prompt suite	golden sets + CI regression + trajectory eval	online guardrail metrics + canary	continuous online eval; auto-gate on regression
Accountability attributable & replayable	opaque logs	structured logs	full audit trail + action attribution	explainable decisions + trace replay	per-action owner; deterministic replay + policy gov
Learning / RL improve from signal	static prompts	manual prompt iteration	offline eval- driven tuning	RLHF / RLAI on curated data	closed-loop from prod traces; verifier rewards
Self-Healing recovery as a property	human fixes everything	alerting + runbooks	detect + suggested fix	semi-auto remediate (propose+approve)	autonomous detect→remediate→ verify w/ guardrails

Increasing maturity →

Figure 2. Axis B — five capability dimensions, each maturing independently from level 1 to 5.

The two axes combine into a single governing rule, and it is the most important sentence in this paper:

An agent's autonomy must never exceed its weakest capability dimension.

A team that grants L4 autonomy while its evaluation capability is still at level 1 ("it looked right") has not built an autonomous agent; it has built an unmonitored one. We call this **autonomy theater**, and it is the single most common and most dangerous anti-pattern in the field. Maturity is earned by strengthening the weakest dimension, not by adding capability the controls cannot yet support.

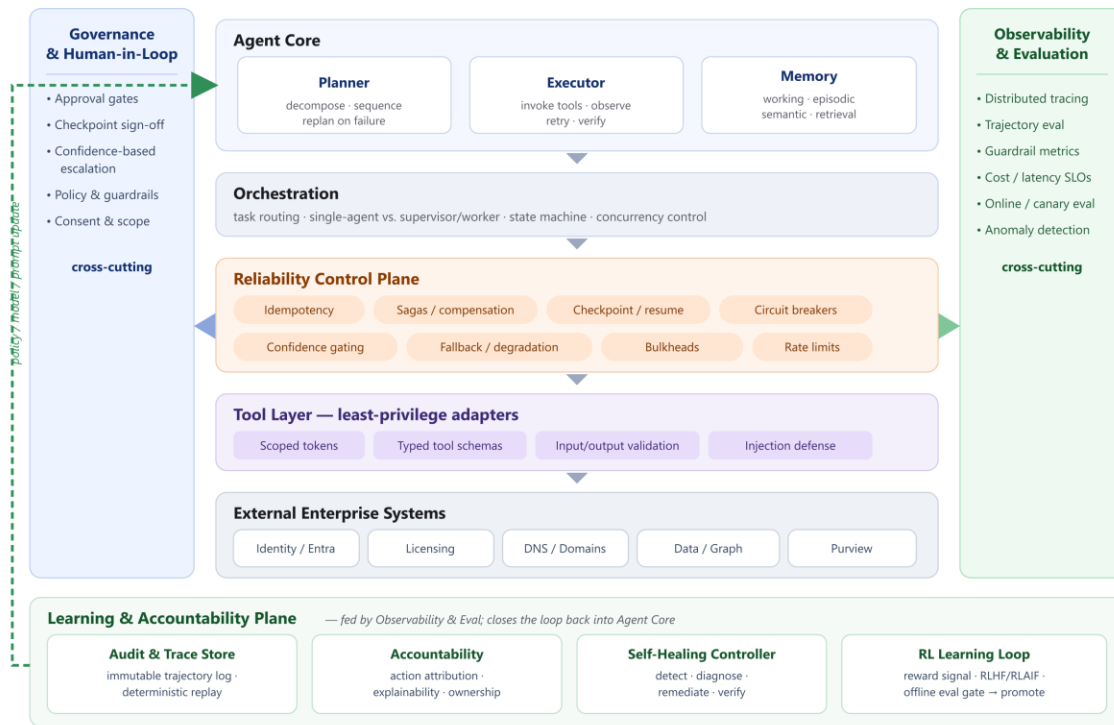
Takeaway. *Maturity is not one number but five, and your safe autonomy ceiling is set by the weakest of them. It rises only when that dimension does — never because the model got better.*

5. A Reference Architecture for Production Agents

If the maturity model says *what* to strengthen, the reference architecture says *where* those strengths live. The design deliberately places the reliability, observability, and learning machinery at the center rather than treating it as an afterthought bolted onto a model call.

Reference Architecture — Production-Grade Agentic AI

Layered control plane for high-stakes, multi-step enterprise workflows (archetype: tenant onboarding / provisioning).



Design principles: idempotency by default · least privilege · observability first · human-gated escalation · bounded blast radius · everything attributable & replayable.

Figure 3. Reference architecture: a layered control plane with cross-cutting governance, observability, and a learning & accountability plane.

The **Agent Core** holds the planner (which decomposes and sequences work, and replans on failure), the executor (which invokes tools, observes results, retries, and verifies), and tiered memory — working, episodic, and semantic — whose design is a first-class reliability concern rather than a prompt trick.³ The core is built on the well-established pattern of interleaving reasoning and acting.⁴

Beneath it, **orchestration** routes tasks and chooses between a single agent and a supervisor/worker decomposition. The **reliability control plane** — the heart of the architecture — supplies idempotency, compensation, checkpoints, circuit breakers, and confidence gating. The **tool layer** exposes only least-privilege, typed, validated adapters to external systems. Two rails run the full height of the system: **governance and human-in-the-loop**, and **observability and evaluation**.

Finally, a **learning and accountability plane** closes the loop. It houses an immutable audit and trace store, the accountability machinery (attribution, explainability, ownership), a self-healing controller, and the reinforcement-learning loop that feeds operational signal back into policy. The dashed feedback path from this plane into the Agent Core is what makes the system *self-improving* rather than static. The guiding principles are constant across layers: idempotency by default, least privilege, observability first, human-gated escalation, bounded blast radius, and everything attributable and replayable.

³Packer et al., “MemGPT,” arXiv:2310.08560, 2023. <https://arxiv.org/abs/2310.08560>

⁴Yao et al., “ReAct,” arXiv:2210.03629, 2022. <https://arxiv.org/abs/2210.03629>

Takeaway. *The model is a component. The reliability, observability, and learning control plane around it is the system that actually ships.*

6. The Reliability Pattern Catalog

The reliability control plane is assembled from a small set of patterns, most borrowed directly from distributed systems and site reliability engineering and re-applied to agents.

- **Idempotent tool calls.** Every state-mutating tool takes an idempotency key so that a retry cannot double-apply an effect — the precondition for safe retries.
- **Compensation and sagas.** A long workflow is modeled as a saga: a sequence of steps, each paired with a compensating action that semantically undoes it, so a mid-flow failure can be unwound instead of abandoned.⁵
- **Checkpoint and resume.** Durable checkpoints let a long-horizon flow resume from the last good state rather than restart, bounding the cost of any single failure.
- **Circuit breakers and bulkheads.** Borrowed from stability engineering, these contain a failing dependency and prevent one bad subsystem from taking down the whole flow.⁶
- **Confidence gating and graceful degradation.** When the agent’s confidence drops below a threshold, it escalates to a human or falls back to a safer, reduced behavior instead of pressing on.

6.1 Reliability Budgeting

The geometric degradation law⁶ is not only a warning; it is a design tool. If a workflow must succeed with probability S over H dependent steps, the required per-step reliability is the H -th root of S . Turning that around gives architects a budget to check *before* committing to an autonomous horizon.

Horizon (steps)	Required per-step reliability for 95% success	Design implication
5	≈ 0.990	Achievable; L3 bounded autonomy is reasonable.
20	≈ 0.997	Demands strong tooling; checkpoint aggressively.
100	≈ 0.9995	Rarely achievable end-to-end; decompose into gated sub-flows.

The lesson is decisive: when the required per-step reliability is unreachable, the correct response is not optimism but **shortening the horizon** — decompose the workflow, insert checkpoints, or add human gates — so that no single autonomous run stretches past what its per-step reliability can support.

⁵Garcia-Molina & Salem, “Sagas,” ACM SIGMOD, 1987. <https://doi.org/10.1145/38713.38742>

⁶Nygaard, Release It!, 2nd ed., 2018. <https://pragprog.com/titles/mnee2/release-it-second-edition/>

7. Quality and Evaluation: The Measurement Backbone

Evaluation is not a phase that follows development; it is the gate that governs every promotion to greater autonomy. You cannot ship, scale, or trust judgment you cannot measure. Mature evaluation has an offline and an online face.

Offline, teams maintain golden datasets, run regression suites in CI, and — critically — score *trajectories*, not just final outputs, because two agents can reach the same answer through wildly different and differently risky paths. Where models judge models, the well-documented biases of LLM-as-judge (position, verbosity, and self-preference) must be measured and corrected rather than trusted,⁷ and judges should be backstopped by deterministic guardrails rather than treated as oracles.⁸

Online, guardrail metrics, canary and shadow deployments, live trajectory sampling, and cost and latency service-level objectives keep a deployed agent honest. Two refinements from recent production practice sharpen this further. First, evaluation should be **horizon-aware**: report success as a function of horizon and per-step reliability, not a single aggregate pass-rate that hides eventual collapse.⁶ Second, evaluation should be **dual-horizon** — assessing both the immediate request trajectory (where in an execution a deficiency arose) and the longer interaction horizon (whether service stays aligned across repeated engagements over time).⁹ To make continuous evaluation affordable, a practical pattern is to *calibrate* an LLM judge against high-confidence ground-truth logs and then *distill* its behavior into a small, low-latency diagnostic model.⁷ The result is a measurement backbone cheap enough to run continuously and trustworthy enough to gate autonomy on.

8. Accountability and Auditability

As autonomy rises, the question “who is responsible for what this agent did?” moves from philosophical to operational. A production agent must make every action **attributable** (who or what initiated it, and why), **auditable** (recorded in an immutable trace), **explainable** (with a recoverable decision rationale), **replayable** (so a trajectory can be deterministically reconstructed), and **owned** (a named human or team accountable for the behavior).

Concretely, this means a trace schema that captures, for each step, the decision, the tool call, its inputs and outputs, the model’s confidence, any human approver, and the outcome. Diagnostic signals should carry explicit evidence scopes and decision boundaries so that an auditor — or the self-healing controller — can localize exactly where behavior diverged.⁷ These practices align the system with emerging risk-management guidance for trustworthy AI,¹⁰ and they are inseparable from security: the same trace that proves accountability is what lets you detect and investigate an attack.

Security deserves explicit attention because the tool layer is an attack surface. Retrieved content can carry injected instructions, over-privileged tools invite confused-deputy exploits, and an agent that

⁷Zheng et al., “Judging LLM-as-a-Judge with MT-Bench,” arXiv:2306.05685, 2023. <https://arxiv.org/abs/2306.05685>

⁸“LLM-as-a-Judge Is Not an Oracle,” arXiv:2609.02246, 2026 (recent preprint). <https://arxiv.org/abs/2609.02246>

⁹Chen et al., “ATLAS: Dual-Horizon Diagnostic Evaluation for Industrial Tool-Use Agents,” arXiv:2608.30685, 2026 (recent preprint). <https://arxiv.org/abs/2608.30685>

¹⁰NIST, “AI Risk Management Framework (AI RMF 1.0),” 2023. <https://www.nist.gov/itl/ai-risk-management-framework>

acts on untrusted input can be steered into exfiltration.¹¹ The defenses are least privilege by default, scoped consent, rigorous input and output validation, and an industry-standard risk checklist applied to every tool.¹²

Takeaway. *If you cannot attribute, replay, and explain an action, you have not deployed an agent — you have deployed an untraceable actor.*

9. Learning from Production: Reinforcement Learning Done Safely

Agents that improve only through hand-edited prompts eventually plateau. Production-grade systems compound quality by learning from their own operational signal — but they do so under guardrails, because a learning loop without safeguards optimizes for the wrong thing faster than a static one.

The **signal** is abundant once accountability is in place: execution traces, human approvals and edits, evaluation outcomes, guardrail hits, task success and failure, and downstream customer outcomes. The **methods** are increasingly well understood — reinforcement learning from human feedback,¹³ reinforcement learning from AI feedback,¹⁴ direct preference optimization,¹⁵ reward modeling from preferences,¹⁶ verbal and self-reflective refinement,¹⁷ and, most recently, rubric- and verifier-based rewards that make the reward signal interpretable and harder to game.¹⁸

The **guardrails** are what make this safe: defense against reward hacking, an offline evaluation gate before any policy is promoted, canary and shadow rollout, human-in-the-loop review of reward labels, and hard regression gates. The payoff is a virtuous loop — better evaluation yields a better reward signal, which yields a better policy, which yields better outcomes — and there is now production evidence that closing this loop with replay-based policy improvement and online experimentation produces concurrent gains in quality and business outcomes.⁷ Learning, in other words, is not a research luxury; it is how a mature agent stops repeating its own mistakes.

10. Self-Healing and Semi-Autonomous Recovery

In a mature system, recovery is a property of the system, not a task on a human’s pager. The mechanism is a closed loop.

¹¹Greshake et al., “Indirect Prompt Injection,” arXiv:2302.12173, 2023. <https://arxiv.org/abs/2302.12173>

¹²OWASP, “Top 10 for LLM Applications,” 2023–2025. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>

¹³Ouyang et al., “InstructGPT,” arXiv:2203.02155, 2022. <https://arxiv.org/abs/2203.02155>

¹⁴Bai et al., “Constitutional AI,” arXiv:2212.08073, 2022. <https://arxiv.org/abs/2212.08073>

¹⁵Rafailov et al., “Direct Preference Optimization,” arXiv:2305.18290, 2023. <https://arxiv.org/abs/2305.18290>

¹⁶Christiano et al., “Deep RL from Human Preferences,” arXiv:1706.03741, 2017. <https://arxiv.org/abs/1706.03741>

¹⁷Shinn et al., “Reflexion,” arXiv:2303.11366, 2023. <https://arxiv.org/abs/2303.11366>

¹⁸“A Survey on Rubric-Guided Reinforcement Learning for Language Models,” arXiv:2608.27505, 2026 (recent preprint). <https://arxiv.org/abs/2608.27505>

Closed-Loop Self-Healing & Continuous Learning

Recovery as a system property, not a pager. Each cycle also produces the signal that improves the next.

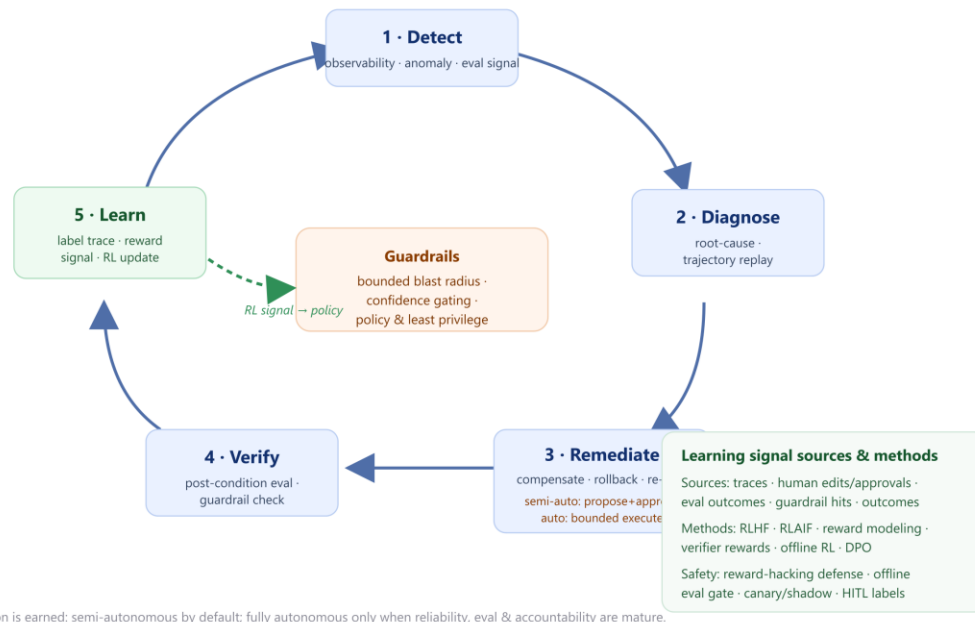


Figure 4. The self-healing loop: detect, diagnose, remediate, verify, learn — with remediation gated by guardrails.

The loop runs in five stages. **Detect** draws on observability, anomaly detection, and live evaluation signals. **Diagnose** performs root-cause analysis and trajectory replay, using diagnostic signals with explicit evidence scopes to localize *where* a failure originated — turning “it failed” into “step seven’s licensing call failed because of X,” which is precisely what makes automated remediation safe.⁷ **Remediate** applies a compensating action, a rollback, or a re-plan. **Verify** confirms recovery against post-conditions and guardrails. **Learn** labels the trace and feeds it back into the reinforcement-learning loop.

The remediation stage has two modes, and the choice between them is governed by the maturity model. In the **semi-autonomous** mode — the safe default for L2–L3 — the system proposes a remediation and a human approves it. In the **autonomous** mode, earned only at L4–L5, the system executes remediation within a bounded blast radius and escalates when its confidence is low. The operational goal, borrowed from site reliability engineering, is a steadily falling mean-time-to-recovery as more of the loop is safely automated.¹⁹ Recovery autonomy, like every other kind, is earned by the maturity of reliability, evaluation, and accountability — never assumed.

11. Anti-Patterns and Trade-offs: When Not To

A framework earns trust by being honest about its own limits. The following are the failure modes we see most often.

- **Autonomy theater** — granting autonomy beyond the weakest capability dimension. The cardinal sin the maturity model exists to prevent.

¹⁹Beyer et al., Site Reliability Engineering, Google, 2016. <https://sre.google/books/>

- **Evaluation by vibes** — shipping judgment because it “looked right,” with no offline gate and no online guardrail.
- **The aggregate pass-rate illusion** — trusting a high benchmark score that hides production collapse as horizons lengthen.⁶
- **The context-truncation trap** — shrinking the context window to save tokens, which *steepens* long-horizon decay because degradation is driven by step count, not context length. Decompose the horizon instead.⁶
- **Premature multi-agent** — reaching for a swarm before a single well-instrumented agent is exhausted. Multi-agent architectures can excel on breadth-first, parallelizable work, but they consume dramatically more tokens — on the order of fifteen times a single chat in one reported system — and become fragile when agents must share context or depend on one another.²⁰ The contrasting practitioner view, that un-shared context makes such systems brittle, is worth weighing seriously before adopting one.²¹
- **Unbounded tools and memory hoarding** — over-privileged tools and ever-growing context that degrade both safety and quality.

The unifying trade-off beneath all of these is that autonomy, breadth, and speed are purchased with cost, latency, and risk. Senior judgment is knowing when that purchase is worth making — and when the simpler design is the better one.²²

12. A Worked Example: Onboarding from L1 to L4

Return to the onboarding agent from Section 2, and trace how the framework matures it. At **L1**, the agent proposes each provisioning step and a human approves every action; nothing is automated, but nothing can go wrong unattended. Moving to **L2**, the team makes every tool idempotent and gates only the state-changing calls, so the agent executes routine steps while a human still authorizes writes.

Reaching **L3** requires the reliability plane: the workflow is modeled as a saga with compensating actions, durable checkpoints are added between milestones, and circuit breakers protect each external dependency. The agent now runs the full flow within guardrails, pausing for human sign-off at milestones, and a mid-flow failure unwinds cleanly instead of stranding the tenant.

Only after evaluation, accountability, and self-healing are demonstrably in place does the team grant **L4**: horizon-aware evaluation gates each release, every action is attributable and replayable, and the self-healing controller detects, diagnoses, and — within a bounded blast radius — remediates common failures automatically, escalating the rest. The measurable results of this progression are the metrics that matter in production: a lower failure rate, a shorter mean-time-to-recovery, less human touch per onboarding, higher evaluation coverage, and, ultimately, the trust that lets the organization widen the agent’s remit. Note that autonomy rose only as fast as the weakest capability dimension allowed — exactly as the governing rule prescribes.

²⁰Anthropic, “How We Built Our Multi-Agent Research System,” 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>

²¹Cognition, “Don’t Build Multi-Agents,” 2025. <https://cognition.ai/blog/dont-build-multi-agents>

²²Anthropic, “Building Effective Agents,” 2024. <https://www.anthropic.com/engineering/building-effective-agents>

13. Getting Started: A Maturity Self-Assessment

Use the scorecard below to locate your system. Score each capability dimension from 1 (ad hoc) to 5 (optimizing). Your **autonomy ceiling** is set by your *lowest* score — that is the maximum autonomy you can safely grant today. The gap between your current autonomy and that ceiling is your risk; the path forward is to raise the weakest dimension before adding autonomy.

Capability dimension	Ask yourself	Score (1–5)
Reliability	Can a mid-flow failure be unwound or resumed without corrupting state?	
Evaluation / Quality	Do we gate releases on horizon-aware, trajectory-level evaluation?	
Accountability	Is every action attributable, auditable, and deterministically replayable?	
Learning / RL	Do we improve from production signal under guardrails, not just prompt edits?	
Self-Healing	Does the system detect, diagnose, and recover — or does a human always?	

The discipline this framework asks for is not glamorous. It is idempotency keys and compensating actions, trajectory evaluation and audit traces, reward guardrails and blast-radius limits. But it is exactly this unglamorous engineering — not the next increment of model capability — that separates an agent that demos from an agent that ships. Autonomy is not granted; it is earned, one capability dimension at a time.

14. The Companion Series: Building Agents That Ship

This whitepaper sets the framework; an eight-part companion series takes it into practice. Each installment isolates one capability dimension and follows the same arc — an anonymized failure story, the mechanism that prevents it, an honest account of when not to apply it, and a reusable diagram or checklist. Read on their own they are field notes; read against the maturity model above they are its operating manual. The map below is the recommended reading order and shows which section of this paper each part deepens.

Part 2 — Reliability & Safe Failure. State-mutating agents need transactional guarantees the model does not give you. Deepens the distributed-transaction reframe (Section 2) and the reliability pattern catalog (Section 6).

Part 3 — The Evaluation Gap. You cannot ship judgment you cannot measure. Deepens the measurement backbone (Section 7).

Part 4 — Accountability & Auditability. Every agent action must be attributable and replayable. Deepens accountability and auditability (Section 8).

Part 5 — Context & Memory. Long-horizon agents live or die on state, not prompts. Deepens tiered memory as a reliability concern (Section 5).

Part 6 — Multi-Agent: Help vs. Hurt. Most multi-agent systems are premature. Deepens orchestration (Section 5) and the premature-multi-agent trap (Section 11).

Part 7 — Learning from Production (RL). Agents that only tune prompts plateau. Deepens safe reinforcement learning (Section 9).

Part 8 — Self-Healing Systems. Recovery should be a system property, not a pager. Deepens the self-healing loop (Section 10).

Part 9 — Securing the Tool Surface (Finale). Every tool is an attack surface. Deepens the security of the tool layer (Section 8).

The through-line across all nine pieces is the single rule this paper is built on: an agent's autonomy must never exceed its weakest capability dimension. The framework tells you what to strengthen; the series shows you how. Start wherever your weakest dimension is today.

Selected References

- [1] H. Garcia-Molina and K. Salem, “Sagas,” ACM SIGMOD, 1987. <https://doi.org/10.1145/38713.38742>
- [2] M. Nygard, Release It! Design and Deploy Production-Ready Software, 2nd ed., 2018. <https://pragprog.com/titles/mnee2/release-it-second-edition/>
- [3] B. Beyer et al., Site Reliability Engineering, Google / O’Reilly, 2016. <https://sre.google/books/>
- [4] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” arXiv:2210.03629, 2022. <https://arxiv.org/abs/2210.03629>
- [5] Anthropic, “Building Effective Agents,” 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- [6] “How Fast Do Agents Rot? An Empirical Study of Long-Horizon Degradation in LLM Agents for Production Decision-Making,” arXiv:2609.01660, 2026. <https://arxiv.org/abs/2609.01660>
- [7] W. Chen et al., “ATLAS: Dual-Horizon Diagnostic Evaluation for Industrial Tool-Use Agents,” arXiv:2608.30685, 2026. <https://arxiv.org/abs/2608.30685>
- [8] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” arXiv:2306.05685, 2023. <https://arxiv.org/abs/2306.05685>
- [9] “LLM-as-a-Judge Is Not an Oracle: Why Self-Improving Agents Need Deterministic Guardrails,” arXiv:2609.02246, 2026. <https://arxiv.org/abs/2609.02246>
- [10] L. Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback (InstructGPT),” arXiv:2203.02155, 2022. <https://arxiv.org/abs/2203.02155>
- [11] Y. Bai et al., “Constitutional AI: Harmlessness from AI Feedback,” arXiv:2212.08073, 2022. <https://arxiv.org/abs/2212.08073>
- [12] R. Rafailov et al., “Direct Preference Optimization,” arXiv:2305.18290, 2023. <https://arxiv.org/abs/2305.18290>
- [13] P. Christiano et al., “Deep Reinforcement Learning from Human Preferences,” arXiv:1706.03741, 2017. <https://arxiv.org/abs/1706.03741>
- [14] N. Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” arXiv:2303.11366, 2023. <https://arxiv.org/abs/2303.11366>
- [15] C. Packer et al., “MemGPT: Towards LLMs as Operating Systems,” arXiv:2310.08560, 2023. <https://arxiv.org/abs/2310.08560>
- [16] K. Greshake et al., “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” arXiv:2302.12173, 2023. <https://arxiv.org/abs/2302.12173>
- [17] OWASP, “Top 10 for LLM Applications,” 2023–2025. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [18] NIST, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
- [19] SAE International, “J3016: Taxonomy and Definitions for Terms Related to Driving Automation Systems,” https://www.sae.org/standards/content/j3016_202104/
- [20] Cognition, “Don’t Build Multi-Agents,” 2025. <https://cognition.ai/blog/dont-build-multi-agents>
- [21] Anthropic, “How We Built Our Multi-Agent Research System,” 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- [22] “A Survey on Rubric-Guided Reinforcement Learning for Language Models,” arXiv:2608.27505, 2026. <https://arxiv.org/abs/2608.27505>

Note: entries marked arXiv:26xx are recent preprints and have not been peer-reviewed; verify details before formal citation. This framework is vendor-neutral; the maturity model is inspired by, and does not reproduce, SAE J3016 and CMMI.